

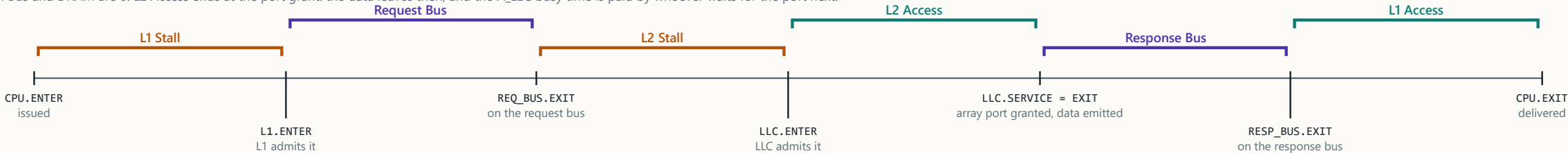
One request, end to end — and what each LatencyReport column measures

Every column is the distance between two Logger events on the request's own timeline, and the columns tile exactly to Total. Ticks are in order, not to scale.
request: GetS / GetM out

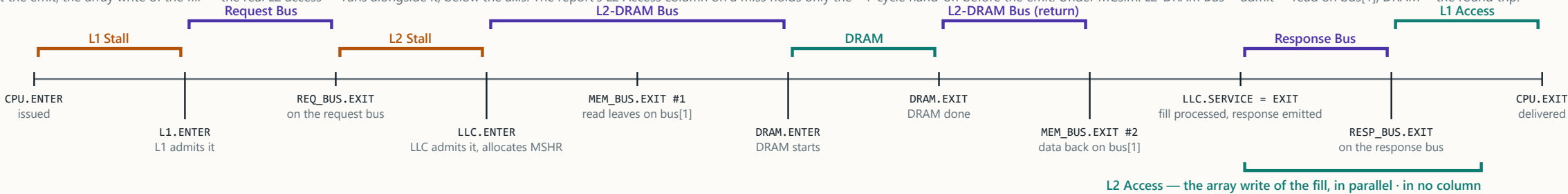


The Logger stamps ENTER / SERVICE / EXIT per component as the request passes; the LatencyReport is computed from those stamps alone.

LLC hit
Six columns; L2-DRAM Bus and DRAM are 0. L2 Access ends at the port grant: the data leaves then, and the A_LLC busy time is paid by whoever waits for the port next.



LLC miss, MainMemoryController
The response leaves at the emit; the array write of the fill — the real L2 access — runs alongside it, below the axis. The report's L2 Access column on a miss holds only the ~1-cycle hand-off before the emit. Under MCsim: L2-DRAM Bus = admit → read on bus[1], DRAM = the round trip.



The columns not on the timelines, and the cases that bend it

- CPU Latency**: issue – the trace timestamp: how long the request waited in the core before it could issue (the out-of-order window).
- Total**: delivered – issued, the sum of the stage columns. Effective: the part of Total not overlapped with the previous request — delivered – max(previous retire, issue).
- L1 hit**: only CPU.ENTER, L1.ENTER and CPU.EXIT exist: L1 Stall and L1 Access, everything else 0.
- cache-to-cache supply**: an owning L1 answers instead of the LLC: its L1.SERVICE / EXIT stand in for the LLC's, so L2 Stall is the wait for the owner and L2 Access its service.
- no responder at all**: admit → grant is unattributable and is folded into L2 Access; the run's stderr counts it as noresp so the gap stays visible.

waiting: in a queue, for a bus, for the port moving: a bus transfer being served: an array or DRAM access

Logger.cpp, "Report: the event timeline is the sole decomposition" · OCTOPUS_EVENT_DUMP=N prints these timelines for the first N slow requests · the [EVENT-PATH] check verifies the tiling on every run